



UNIVERSITY OF
MAURITIUS

2026

MARIE SIMULATOR

CRAMER'S RULE

ICT 1206Y Computer Architecture and Organisation

Prepared by:

Makhun Lokeshav Anand

2514204

Moonesawmy Seren

2516246

Gowreesunkur Baboo Jatinsingh

2516213

Submission Date :
3rd April 2026

TABLE OF CONTENTS

1. Problem Statement	2
2. Flowchart	3
3. Subroutines Overview	4
Multiplication Subroutine:	4
Division Subroutine:	10
4. Test Cases.....	16
Test Case 1: Unique Solution ($D \neq 0$)	16
Test Case 2: Non-Unique Solution ($D = 0$)	18
Test Case 3: Rounding Decimal Values	20
5. Report / Documentation	22
Purpose of the program	22
How is Cramer's Rule implemented:.....	23
How are MARIE instructions used:.....	24
Limitations in MARIE simulator:.....	25
Reflection on what we learned	26
CONCLUSION	27
REFERENCES	28

1. Problem Statement

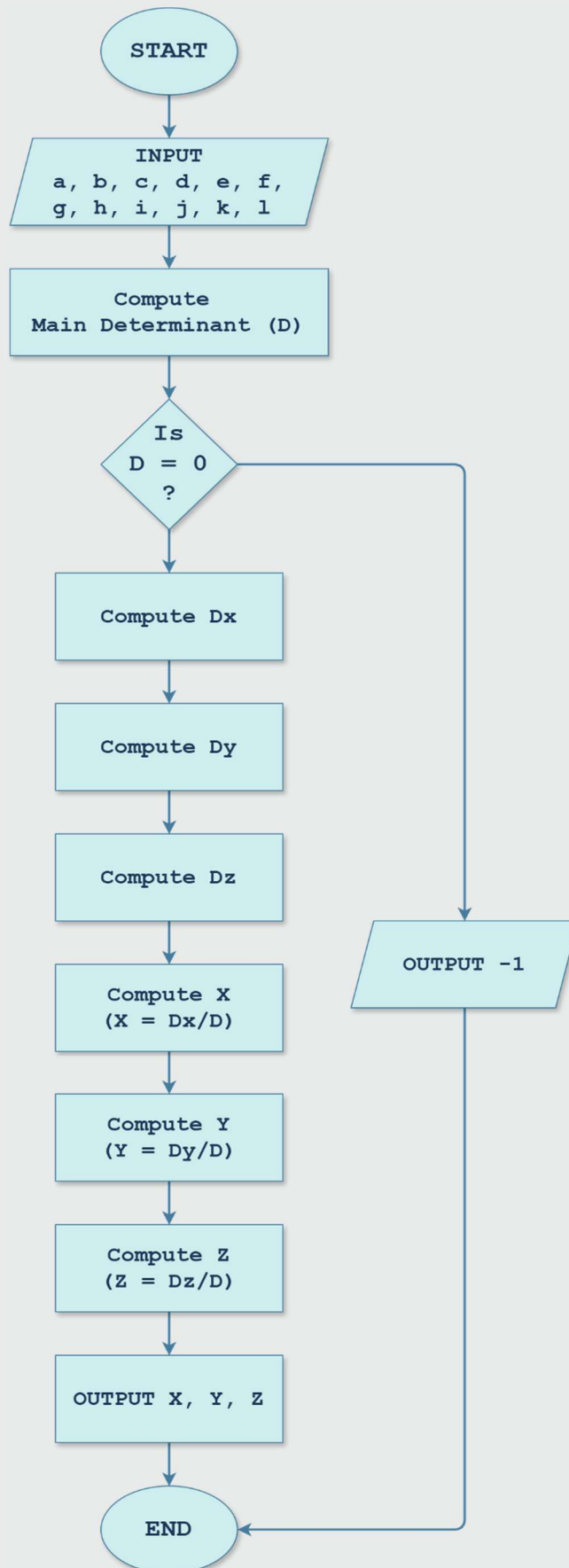
This program solves a system of three simultaneous linear equations with three unknowns (x, y, z) using Cramer's Rule, implemented entirely in MARIE assembly language.

The general form of the system is:

$$\begin{array}{rclclclclcl} \textcolor{red}{a} \cdot x & + & \textcolor{red}{b} \cdot y & + & \textcolor{red}{c} \cdot z & = & \textcolor{red}{d} \\ \textcolor{red}{e} \cdot x & + & \textcolor{red}{f} \cdot y & + & \textcolor{red}{g} \cdot z & = & \textcolor{red}{h} \\ \textcolor{red}{i} \cdot x & + & \textcolor{red}{j} \cdot y & + & \textcolor{red}{k} \cdot z & = & \textcolor{red}{l} \end{array}$$

The program accepts 12 integer inputs (a, b, c, d, e, f, g, h, i, j, k, l) in order, computes the main determinant D and the modified determinants Dx, Dy, Dz, then outputs the integer solutions for x, y, and z. If D = 0, the program outputs -1 and terminates.

2. Flowchart



3. Subroutines Overview

This section provides detailed explanations and walkthroughs of the two arithmetic subroutines implemented in MARIE assembly language: **Multiplication** and **Division**. Since MARIE does not natively support either operation, both subroutines must be designed and written using only the basic MARIE instruction set. Hence, each subroutine is broken down into its individual labelled sections, with supporting screenshots and purpose explained in detail.

Multiplication Subroutine:

This subroutine performs integer multiplication of **three variables** using **repeated addition** - the fundamental approach available in MARIE's instruction set. Since MARIE lacks a native multiply instruction, the subroutine simulates multiplication by adding one operand to a running total several times equal to the other operand.

To handle **negative numbers**, a sign-tracking variable is maintained throughout the subroutine. To optimise **performance**, the operands are compared and swapped so that the larger value is added fewer times. For **three-variable multiplication**, the subroutine calls itself using a Boolean flag to control termination.

➤ Variables

Variable Name	Purpose
<code>parameter1</code>	First operand (the value to be repeatedly added)
<code>parameter2</code>	Second operand (acts as the loop counter)
<code>parameter3</code>	Third operand (used in conjunction with <code>parameter3Swap</code>)
<code>tempResult</code>	Accumulated the running multiplication result
<code>sign_mult</code>	Tracks the sign of the result (0 = positive, 1 = negative, 2 = positive)
<code>parameter3Swap</code>	Acts as a Boolean flag indicating whether a third variable still needs to be processed. (0 or 1)
<code>SwapTemp</code>	Temporary storage used during the operand swap

➤ Subroutine Walkthrough:

1) Multiply

```
506 /Multiplication subroutine (3 Variables Multiplication)
507
508 Multiply,  HEX 0
509         LOAD ONE /Initialising the "boolean" parameter3Swap
510         STORE parameter3Swap
511
```

Figure 1 — Multiply entry point

The label **Multiply** serves as the subroutine's entry point. When called via **JnS Multiply**, MARIE automatically stores the return address (the address of the instruction following the call) into the memory location labelled **Multiply**. This ensures that once the subroutine finishes, execution resumes correctly in the calling program via **JUMPI Multiply**.

Immediately after entry, **parameter3Swap** is initialised to **1**, signalling that a third variable is to be processed. This flag is only set here (at the true entry point), not at **Initialise**, which is important for the three-variable logic described later.

2) Initialise

```
512 Initialise, LOAD ZERO
513         STORE sign_mult
514         STORE tempResult
515         LOAD parameter1
516         SKIPCOND 400
517         JUMP CheckNum1
518         JUMP OutputZero
519
```

Figure 2 — Initialise

Each multiplication pass begins by resetting **sign_mult** and **tempResult** to zero, clearing any values from a previous pass. Then, **parameter1** is examined: if it equals zero, the result is immediately zero and execution jumps to **OutputZero**. Otherwise, processing continues to **CheckNum1**.

3) CheckNum1

```
520 CheckNum1, SKIPCOND 000
521         JUMP CheckNum2
522         LOAD sign_mult
523         ADD ONE
524         STORE sign_mult
525         LOAD ZERO
526         SUBT parameter1
527         STORE parameter1
528         JUMP CheckNum2
```

Figure 3 — CheckNum1

CheckNum1 inspects **parameter1**. If it is negative, **sign_mult** is incremented by 1 and the operand is converted to its absolute value using **0 - parameter1**. Execution then falls through to **CheckNum**.

4) CheckNum2

```
531 CheckNum2, LOAD parameter2
532         SKIPCOND 400
533         JUMP CheckNum2_1
534         JUMP OutputZero
535
```

Figure 4 — CheckNum2

CheckNum2 first checks whether **parameter2** is zero — if so, the result is zero and the subroutine jumps to **OutputZero**. Else, the process continues with **CheckNum2_1**.

5) CheckNum2_1

```
536 CheckNum2_1, SKIPCOND 000
537         JUMP CheckGreatest
538         LOAD sign_mult
539         ADD ONE
540         STORE sign_mult
541         LOAD ZERO
542         SUBT parameter2
543         STORE parameter2
544         JUMP CheckGreatest
```

Figure 5 — CheckNum2_1

CheckNum2_1 is the second phase of the above step. It first checks if **parameter2** is negative. If so, **sign_mult** is incremented, performs the same negative-number handling on **parameter2** as **CheckNum1** did on **parameter1** and jump to **CheckGreatest**. If not, skip to **CheckGreatest**.

6) CheckGreatest

```
546 CheckGreatest, LOAD parameter1
547                SUBT parameter2
548                SKIPCOND 400
549                SKIPCOND 000
550                JUMP Multiplication
551                JUMP Swap
552
```

Figure 6 — CheckGreatest

CheckGreatest determines which is greater between **parameter1** and **parameter2**. The comparison is performed by subtracting **parameter2** from **parameter1**. If the result is zero or positive (i.e., **parameter1** $\geq$ **parameter2**), no swap is needed. If the result is negative, the program jumps to **Swap**.

7) Swap

```
553 Swap,        LOAD parameter2
554                STORE SwapTemp
555                LOAD parameter1
556                STORE parameter2
557                LOAD SwapTemp
558                STORE parameter1
559
```

Figure 7 — Swap

Swap exchanges the values between **parameter1** and **parameter2** using **SwapTemp** as a temporary holding variable, to ensure **parameter1** is the greatest.

8) Multiplication

```
560 Multiplication, LOAD tempResult
561                ADD parameter1
562                STORE tempResult
563                LOAD parameter2
564                SUBT ONE
565                STORE parameter2
566                SKIPCOND 400
567                JUMP Multiplication
568                JUMP OutputResult
569
```

Figure 8 — Multiplication

This is the heart of the subroutine. On each iteration:

1. **parameter1** is added to **tempResult**.
2. **parameter2** (the counter) is decremented by 1.
3. If **parameter2** has reached zero, the loop exits to **OutputResult**. Otherwise, it repeats the 2 steps above.

By the time the loop ends, **tempResult** holds the product of the two (now positive) operands.

9) OutputResult

```

570 OutputResult, LOAD sign_mult
571             SUBT ONE
572             SKIPCOND 400
573             JUMP CheckNum3
574             LOAD ZERO
575             SUBT tempResult
576             STORE tempResult
577             JUMP CheckNum3
578

```

Figure 9 — OutputResult

Once multiplication is complete, **sign_mult** is checked. The value is decremented by 1 and tested for zero: if the result is zero (meaning **sign_mult** was originally 1), the product must be negative, so **tempResult** becomes negative using **0 - tempResult**. If **sign_mult** was 0 or 2, the result is already positive and no change is made.

10) CheckNum3

```

579 CheckNum3, LOAD parameter3Swap
580             SKIPCOND 800
581             JUMPI Multiply /All three parameters have been multiplied
582             SUBT ONE
583             STORE parameter3Swap
584             LOAD parameter3
585             STORE parameter1
586             LOAD tempResult
587             STORE parameter2
588             JUMP Initialise /Loop back to start comparing & multiplying
589

```

Figure 10 — CheckNum3

After processing two operands, the subroutine checks whether a third variable exists by inspecting **parameter3Swap**.

- If **parameter3Swap** is **0**, all multiplication is complete and the subroutine returns via **JUMPI Multiply** (jumping to the return address saved at entry).
- If **parameter3Swap** is **positive** (**=1**), there is still a third operand to process. **parameter3Swap** is decremented to 0 (preventing a third recursive pass), **parameter3** is loaded into **parameter1**, and **tempResult** (the result so far) is loaded into **parameter2**. The subroutine then jumps back to **Initialise** — **not Multiply** — to avoid resetting **parameter3Swap** a second time.

11) OutputZero

```

590 OutputZero, LOAD ZERO
591             STORE tempResult
592             JUMPI Multiply

```

Figure 11 — OutputZero

If either **parameter1** or **parameter2** is zero at any point, the result is trivially zero. **tempResult** is set to 0 and the subroutine returns immediately.

➤ Key Points:

- How does `sign_mult` handle negative numbers?

Negative numbers cannot be used directly in repeated addition, so the subroutine converts both operands to their absolute values before multiplying. The sign of the result is tracked separately using `sign_mult`, which accumulates a value based on the signs of the two operands:

parameter1	parameter2	sign_mult	Result Sign
Positive	Positive	0	Positive
Negative	Positive	1	Negative
Positive	Negative	1	Negative
Negative	Negative	2	Positive

The pattern is straightforward: whenever `sign_mult` equals 1, the result must be negated. Values of 0 or 2 both indicate a positive result. Hence, `sign_mult` is checked after the multiplicative addition is over, to determine the polarity of the result.

- Why is `Swap` used for optimization?

Since the multiplication loop runs `parameter2` times (adding `parameter1` on each iteration), it is more efficient to ensure `parameter1` is the **larger** value and `parameter2` is the **smaller** one. This minimises the number of loop iterations.

For example, computing 3×10 by adding 3 ten times requires 10 iterations. Swapping to compute 10×3 — adding 10 three times — requires only 3 iterations, a significant reduction for large inputs.

Note: The impact of this modification is only meaningful when the two operands differ significantly in magnitude. For small or similarly sized numbers, the time saved by fewer iterations is negligible and may not justify the additional lines of code introduced by the swap logic. Nevertheless, for larger inputs where the difference between operands is substantial — for example, multiplying 3×1000 versus 1000×3 — the reduction in iterations is considerable, making the swap a worthwhile trade-off. On balance, the added code complexity is a fixed cost, while the performance gain scales with input size, so the optimisation remains a net benefit for general-purpose use.

Division Subroutine:

This subroutine performs **integer division** of two numbers in MARIE using **repeated subtraction** - the fundamental approach available within MARIE's instruction set. Since MARIE lacks a native divide instruction, the subroutine simulates division by repeatedly subtracting the divisor from the dividend, counting how many times the subtraction can be performed before the result turns negative. That count becomes the **quotient**.

The subroutine also handles **negative operands** using a sign-tracking variable and applies **standard rounding** to the quotient based on the size of the remainder relative to the divisor.

➤ Variables

Variable Name	Purpose
dividend	The number being divided (numerator)
divisor	The number to divide by (denominator)
quotient	Accumulates the result of the division, incremented on each successful subtraction
remainder	Stores the running result of each subtraction; also used to determine rounding
sign_div	Tracks the sign of the final result (0 or 2 = positive; 1 = negative)

➤ Subroutine Walkthrough:

1) Division

431	/DIVISION SUBROUTINE
432	Division, HEX 0
433	LOAD ZERO
434	STORE quotient
435	STORE sign_div
436	LOAD dividend
437	SKIPCOND 000
438	JUMP Checkdivisor
439	LOAD sign_div
440	ADD ONE
441	STORE sign_div
442	LOAD ZERO
443	SUBT dividend
444	STORE dividend
445	JUMP Checkdivisor

Figure 1 —Division entry point

Division is the subroutine's entry point. When called via **JnS Division**, MARIE automatically stores the return address into the memory location labelled **Division**, ensuring that execution resumes correctly in the calling program via **JUMPI Division** at the end. Upon entry, both **quotient** and **sign_div** are immediately initialised to zero.

The **dividend** is then loaded and checked for sign using **SKIPCOND 000**. If it is positive or zero, execution skips directly to **Checkdivisor**. If it is negative, **sign_div** is incremented by 1 and the **dividend** is converted to its absolute value using **0 - dividend** before proceeding to **Checkdivisor**.

2) Checkdivisor

447	Checkdivisor, LOAD divisor
448	SKIPCOND 000
449	JUMP Divide
450	LOAD sign_div
451	ADD ONE
452	STORE sign_div
453	LOAD ZERO
454	SUBT divisor
455	STORE divisor
456	JUMP Divide

Figure 2 — Initialise

Checkdivisor loads **divisor** which is then checked with **SKIPCOND 000**. If it is **positive or zero**, execution jumps directly to **Divide**. If it is **negative**, **sign_div** is incremented by 1 and the **divisor** is converted to its absolute value using **0 - divisor**. Execution then proceeds to **Divide**.

After both checks, all further computation operates exclusively on positive integers, with **sign_div** carrying full responsibility for the sign of the final result.

3) Divide

458	Divide, LOAD dividend
459	STORE remainder

Figure 3 — CheckNum1

Before the subtraction loop can begin, the value of the **dividend** is copied into remainder. This sets up **remainder** as the running value that will be progressively reduced by the **divisor** on each loop iteration.

4) Loop and Continue_2

461	Loop, LOAD remainder
462	SUBT divisor
463	SKIPCOND 000
464	JUMP Continue_2
465	JUMP Done
467	Continue_2, STORE remainder
468	LOAD quotient
469	ADD ONE
470	STORE quotient
471	JUMP Loop

Figure 4 — CheckNum2

Loop and **Continue_2** are the core of the division process. On each iteration of the loop, the **divisor** is subtracted from **remainder** and the result is tested:

- If the result is negative, implying that the divisor is larger than what remains, then division is complete. The result is not stored into remainder (preserving the last valid remainder for rounding). Execution jumps to **Done**.
- If the result is zero or positive, then the subtraction was successful. Execution continues to **Continue_2**, which stores the new value back into remainder, increments quotient by 1, and jumps back to **Loop** to repeat the process.

By the time the loop terminates, **quotient** holds the integer result of the division and **remainder** holds the leftover value from the last successful subtraction.

5) Done

```
473 Done,  LOAD remainder
474        SKIPCOND 800
475        JUMP Continue_3
476        STORE parameter1
477        LOAD TWO
478        STORE parameter2
479        LOAD ONE
480        STORE parameter3
481        JnS Multiply
482        LOAD tempResult
483        SUBT divisor
484        SKIPCOND 000
485        JnS Round_up
```

Figure 6 — CheckGreatest

Once the loop ends, **remainder** is loaded and checked using **SKIPCOND 800**.

If remainder is **zero**, the division was exact — no rounding is required and execution jumps straight to **Continue_3**.

If remainder is **positive** (non-zero), rounding logic is applied:

The subroutine calls the **Multiply subroutine** to double the remainder, passing **remainder** as **parameter1**, **2** as **parameter2**, and **1** as **parameter3** (Remainder x 2 x 1). The result is stored in **tempResult**. The **divisor** is then subtracted from **tempResult** and the result is tested using **SKIPCOND 000**:

- If **remainder** × 2 ≥ **divisor** (result is zero or positive) → **JnS Round_up** is called, adding 1 to the **quotient**.
- If **remainder** × 2 < **divisor** (result is negative) → the **SKIPCOND 000** skips the **JnS Round_up** call and the **quotient** remains unchanged.

Execution then falls through to **Continue_3**.

6) Continue_3

```
487 Continue_3, LOAD sign_div
488           SUBT ONE
489           SKIPCOND 400
490           JUMP Posquotient
491           LOAD ZERO
492           SUBT quotient
493           JUMPI Division
```

Figure 7 — Swap

Continue_3 determines the final sign of the quotient. **sign_div** is decremented by 1 and the result is tested using **SKIPCOND 400**:

- If the result is **not zero** (meaning **sign_div** was 0 or 2) → execution jumps to **Posquotient**
- If the result is **zero** (meaning **sign_div** was originally 1) → the **quotient** is negative. It is computed as **0 - quotient**, loaded into the accumulator, and the subroutine returns via **JUMPI Division**.

In both cases, the final quotient is held in the **accumulator** upon return, making it immediately accessible to the rest of the program.

7) Round_up

```
495 Round_up, HEX 0
496           LOAD quotient
497           ADD ONE
498           STORE quotient
499           JUMPI Round_up
```

Figure 8 — Multiplication

Round_up is itself a small subroutine, entered via **JnS Round_up**. It simply adds 1 to **quotient** and returns to the calling point inside **Done** via **JUMPI Round_up** — execution then falls naturally into **Continue_3**.

8) Posquotient

```
501 Posquotient, LOAD quotient
502           JUMPI Division
503
```

Figure 9 — OutputResult

Posquotient loads the positive quotient into the accumulator and returns to the calling program via **JUMPI Division**.

➤ Key Points:

- How does **sign_div** handle negative numbers?

Negative numbers cannot be used directly in repeated subtraction, so the subroutine converts both operands to their absolute values before division. The sign of the result is tracked separately using **sign_div**, which accumulates a value based on the signs of the two operands:

parameter1	parameter2	sign_mult	Result Sign
Positive	Positive	0	Positive
Negative	Positive	1	Negative
Positive	Negative	1	Negative
Negative	Negative	2	Positive

The pattern is straightforward: whenever **sign_div** equals 1, the result must be negated. Values of 0 or 2 both indicate a positive result. Hence, **sign_div** is checked after the result of the repeated subtraction is over, to determine the polarity of the result.

This is the same concept used for the multiply subroutine using **sign_mult**.

- What is the need for **Round_up** in the division?

Since MARIE operates exclusively on integers, decimal numbers cannot be inputted or stored, meaning the result of any division operation must also be expressed as a whole number.

However, dividing two integers does not always yield an exact integer result as a remainder is frequently left over. Simply discarding this remainder would **always round the result down**, which is not always the most accurate representation of the true value. To produce a result that is as close as possible to the mathematically correct answer, **standard rounding** is applied.

The rule used is the **round half up** principle: if the fractional part of the result is greater than or equal to one-half, the quotient is rounded up; otherwise, it is left unchanged (rounded down). This is expressed by the inequality:

$$\frac{\text{Remainder}}{\text{Divisor}} \geq \frac{1}{2}$$

Making divisor the subject of formula, the equation is rewritten as :

$$\text{Remainder} \times 2 \geq \text{Divisor}$$

When the condition is satisfied, **Round_up** subroutine is called and adds one to the quotient. If the condition is not met, the quotient remains unchanged and is outputted, effectively performing a rounding down.

4. Test Cases

Test Case 1: **Unique Solution ($D \neq 0$)**

Field	Details
System of Equations	$4x - y + 2z = 15$ $-x + 2y + 3z = 5$ $5x - 7y + 9z = 8$
Input Values (a-l)	4, -1, 2, 15, -1, 2, 3, 5, 5, -7, 9, 8
Expected Output	$x = 4, y = 3, z = 1$
Pass / Fail	Pass
Comments	The execution of the code takes around 5 minutes to complete.

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC: 0000 (Hex)
IR: 5000 (Hex)
MAR: 100 (Hex)
MBR: 0000 (Hex)
PC: 101 (Hex)
INPUT: 4 Dec

Test Case 1 — Input a

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC: 0004 (Hex)
IR: 5000 (Hex)
MAR: 102 (Hex)
MBR: 0004 (Hex)
PC: 103 (Hex)
INPUT: -1 Dec

Test Case 1 — Input b

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC: FFFF (Hex)
IR: 5000 (Hex)
MAR: 104 (Hex)
MBR: FFFF (Hex)
PC: 105 (Hex)
INPUT: 2 Dec

Test Case 1 — Input c

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC: 0002 (Hex)
IR: 5000 (Hex)
MAR: 106 (Hex)
MBR: 0002 (Hex)
PC: 107 (Hex)
INPUT: 15 Dec

Test Case 1 — Input d

	label	opcode	operand	hex
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f
☐	10C		INPUT	5000
☐	10D		STORE	g
☐	10E		INPUT	5000

AC: 000F (Hex)
IR: 5000 (Hex)
MAR: 108 (Hex)
MBR: 000F (Hex)
PC: 109 (Hex)
INPUT: -1 Dec

Test Case 1 — Input e

	label	opcode	operand	hex
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f
☐	10C		INPUT	5000
☐	10D		STORE	g
☐	10E		INPUT	5000

AC: FFFF (Hex)
IR: 5000 (Hex)
MAR: 10A (Hex)
MBR: FFFF (Hex)
PC: 10B (Hex)
INPUT: 2 Dec

Test Case 1 — Input f

	label	opcode	operand	hex
☐	103	STORE	b	22B5
☐	104	INPUT		5000
☐	105	STORE	c	22B6
☐	106	INPUT		5000
☐	107	STORE	d	22B7
☐	108	INPUT		5000
☐	109	STORE	e	22B8
☐	10A	INPUT		5000
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000

AC 0002 (Hex)
IR 5000 (Hex)
MAR 10C (Hex)
MBR 0002 (Hex)
PC 10D (Hex)
INPUT 3 Dec

Test Case 1 — Input g

	label	opcode	operand	hex
☐	103	STORE	b	22B5
☐	104	INPUT		5000
☐	105	STORE	c	22B6
☐	106	INPUT		5000
☐	107	STORE	d	22B7
☐	108	INPUT		5000
☐	109	STORE	e	22B8
☐	10A	INPUT		5000
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000

AC 0003 (Hex)
IR 5000 (Hex)
MAR 10E (Hex)
MBR 0003 (Hex)
PC 10F (Hex)
INPUT 5 Dec

Test Case 1 — Input h

	label	opcode	operand	hex
☐	105	STORE	c	22B6
☐	106	INPUT		5000
☐	107	STORE	d	22B7
☐	108	INPUT		5000
☐	109	STORE	e	22B8
☐	10A	INPUT		5000
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000
☐	10F	STORE	h	22BB
☐	110	INPUT		5000

AC 0005 (Hex)
IR 5000 (Hex)
MAR 110 (Hex)
MBR 0005 (Hex)
PC 111 (Hex)
INPUT 5 Dec

Test Case 1 — Input i

	label	opcode	operand	hex
☐	107	STORE	d	22B7
☐	108	INPUT		5000
☐	109	STORE	e	22B8
☐	10A	INPUT		5000
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000
☐	10F	STORE	h	22BB
☐	110	INPUT		5000
☐	111	STORE	i	22BC
☐	112	INPUT		5000

AC 0005 (Hex)
IR 5000 (Hex)
MAR 112 (Hex)
MBR 0005 (Hex)
PC 113 (Hex)
INPUT -7 Dec

Test Case 1 — Input j

	label	opcode	operand	hex
☐	109	STORE	e	22B8
☐	10A	INPUT		5000
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000
☐	10F	STORE	h	22BB
☐	110	INPUT		5000
☐	111	STORE	i	22BC
☐	112	INPUT		5000
☐	113	STORE	j	22BD
☐	114	INPUT		5000

AC FFF9 (Hex)
IR 5000 (Hex)
MAR 114 (Hex)
MBR FFF9 (Hex)
PC 115 (Hex)
INPUT 9 Dec

Test Case 1 — Input k

	label	opcode	operand	hex
☐	10B	STORE	f	22B9
☐	10C	INPUT		5000
☐	10D	STORE	g	22BA
☐	10E	INPUT		5000
☐	10F	STORE	h	22BB
☐	110	INPUT		5000
☐	111	STORE	i	22BC
☐	112	INPUT		5000
☐	113	STORE	j	22BD
☐	114	INPUT		5000
☐	115	STORE	k	22BE
☐	116	INPUT		5000

AC 0009 (Hex)
IR 5000 (Hex)
MAR 116 (Hex)
MBR 0009 (Hex)
PC 117 (Hex)
INPUT 8 Dec

Test Case 1 — Input l

	label	opcode	operand	hex
☐	227	STORE	z	22D8
☐	228	LOAD	z	12D8
☐	229	OUTPUT		6000
☐	22A	HALT		7000
☐	22B	Division	HEX 0	0000
☐	22C	LOAD	ZERO	12D9
☐	22D	STORE	quotient	22D3
☐	22E	STORE	sign_div	22D5
☐	22F	LOAD	dividend	12D1
☐	230	SKIPCOND	000	8000
☐	231	JUMP	Checkd...	9239
☐	232	LOAD	sign div	12D5

AC 0001 (Hex)
IR 7000 (Hex)
MAR 22A (Hex)
MBR 0001 (Hex)
PC 22B (Hex)
INPUT 8 Dec

OUTPUT

4
3
1

Dec Control

Test Case 1 — Output x, y, z

Test Case 2: Non-Unique Solution (D = 0)

Field	Details
System of Equations	$1x + 2y + 3z = 10$ $4x + 5y + 6z = 11$ $7x + 8y + 9z = 12$
Input Values (a–l)	1,2,3,10,4,5,6,11,7,8,9,12
Expected Output	-1
Pass / Fail	Pass
Comments	The program ends quicker because as soon as the calculation of D equals to 0, the program terminates with output -1. (Divide by zero error)

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 0001 (Hex)
IR 5000 (Hex)
MAR 102 (Hex)
MBR 0001 (Hex)
PC 103 (Hex)
INPUT 2 Dec

Test Case 2 — Input a

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 0000 (Hex)
IR 5000 (Hex)
MAR 100 (Hex)
MBR 0000 (Hex)
PC 101 (Hex)
INPUT 1 Dec

Test Case 2 — Input b

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 0002 (Hex)
IR 5000 (Hex)
MAR 104 (Hex)
MBR 0002 (Hex)
PC 105 (Hex)
INPUT 3 Dec

Test Case 2 — Input c

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 0003 (Hex)
IR 5000 (Hex)
MAR 106 (Hex)
MBR 0003 (Hex)
PC 107 (Hex)
INPUT 10 Dec

Test Case 2 — Input d

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 000A (Hex)
IR 5000 (Hex)
MAR 108 (Hex)
MBR 000A (Hex)
PC 109 (Hex)
INPUT 4 Dec

Test Case 2 — Input e

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a
☐	102		INPUT	5000
☐	103		STORE	b
☐	104		INPUT	5000
☐	105		STORE	c
☐	106		INPUT	5000
☐	107		STORE	d
☐	108		INPUT	5000
☐	109		STORE	e
☐	10A		INPUT	5000
☐	10B		STORE	f

AC 0004 (Hex)
IR 5000 (Hex)
MAR 10A (Hex)
MBR 0004 (Hex)
PC 10B (Hex)
INPUT 5 Dec

Test Case 2 — Input f

	label	opcode	operand	hex
☐	101	STORE	a	22B4
☐	102	INPUT	5000	
☐	103	STORE	b	22B5
☐	104	INPUT	5000	
☐	105	STORE	c	22B6
☐	106	INPUT	5000	
☐	107	STORE	d	22B7
☐	108	INPUT	5000	
☐	109	STORE	e	22B8
☐	10A	INPUT	5000	
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	

AC 0005 (Hex)
IR 5000 (Hex)
MAR 10C (Hex)
MBR 0005 (Hex)
PC 10D (Hex)
INPUT 6 Dec

Test Case 2 — Input g

	label	opcode	operand	hex
☐	103	STORE	b	22B5
☐	104	INPUT	5000	
☐	105	STORE	c	22B6
☐	106	INPUT	5000	
☐	107	STORE	d	22B7
☐	108	INPUT	5000	
☐	109	STORE	e	22B8
☐	10A	INPUT	5000	
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	
☐	10D	STORE	g	22BA
☐	10E	INPUT	5000	

AC 0006 (Hex)
IR 5000 (Hex)
MAR 10E (Hex)
MBR 0006 (Hex)
PC 10F (Hex)
INPUT 11 Dec

Test Case 2 — Input h

	label	opcode	operand	hex
☐	105	STORE	c	22B6
☐	106	INPUT	5000	
☐	107	STORE	d	22B7
☐	108	INPUT	5000	
☐	109	STORE	e	22B8
☐	10A	INPUT	5000	
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	
☐	10D	STORE	g	22BA
☐	10E	INPUT	5000	
☐	10F	STORE	h	22BB
☐	110	INPUT	5000	

AC 000B (Hex)
IR 5000 (Hex)
MAR 110 (Hex)
MBR 000B (Hex)
PC 111 (Hex)
INPUT 7 Dec

Test Case 2 — Input i

	label	opcode	operand	hex
☐	107	STORE	d	22B7
☐	108	INPUT	5000	
☐	109	STORE	e	22B8
☐	10A	INPUT	5000	
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	
☐	10D	STORE	g	22BA
☐	10E	INPUT	5000	
☐	10F	STORE	h	22BB
☐	110	INPUT	5000	
☐	111	STORE	i	22BC
☐	112	INPUT	5000	

AC 0007 (Hex)
IR 5000 (Hex)
MAR 112 (Hex)
MBR 0007 (Hex)
PC 113 (Hex)
INPUT 8 Dec

Test Case 2 — Input j

	label	opcode	operand	hex
☐	109	STORE	e	22B8
☐	10A	INPUT	5000	
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	
☐	10D	STORE	g	22BA
☐	10E	INPUT	5000	
☐	10F	STORE	h	22BB
☐	110	INPUT	5000	
☐	111	STORE	i	22BC
☐	112	INPUT	5000	
☐	113	STORE	j	22BD
☐	114	INPUT	5000	

AC 0008 (Hex)
IR 5000 (Hex)
MAR 114 (Hex)
MBR 0008 (Hex)
PC 115 (Hex)
INPUT 9 Dec

Test Case 2 — Input k

	label	opcode	operand	hex
☐	10B	STORE	f	22B9
☐	10C	INPUT	5000	
☐	10D	STORE	g	22BA
☐	10E	INPUT	5000	
☐	10F	STORE	h	22BB
☐	110	INPUT	5000	
☐	111	STORE	i	22BC
☐	112	INPUT	5000	
☐	113	STORE	j	22BD
☐	114	INPUT	5000	
☐	115	STORE	k	22BE
☐	116	INPUT	5000	

AC 0009 (Hex)
IR 5000 (Hex)
MAR 116 (Hex)
MBR 0009 (Hex)
PC 117 (Hex)
INPUT 12 Dec

Test Case 2 — Input l

	label	opcode	operand	hex
☐	14F	SUBT	var2	42C8
☐	150	SUBT	var3	42C9
☐	151	ADD	var4	32CA
☐	152	ADD	var5	32CB
☐	153	SUBT	var6	42CC
☐	154	STORE	D	22CD
☐	155	LOAD	D	12CD
☐	156	SKIPCOND	400	8400
☐	157	JUMP	Continue	915B
☐	158	LOAD	ERROR	12DC
☐	159	OUTPUT		6000
☐	15A	HALT		7000

AC FFFF (Hex)
IR 7000 (Hex)
MAR 15A (Hex)
MBR FFFF (Hex)
PC 15B (Hex)
INPUT 12 Dec

OUTPUT
-1
Dec Control

Test Case 2 — Output -1

Test Case 3: Rounding Decimal Values

Field	Details
System of Equations	$3x + 2y + z = 14$ $x + 4y + 2z = 18$ $2x + 3y + 5z = 25$
Input Values (a–i)	3,2,1,14,1,4,2,18,2,3,5,25
Expected Output	$x = 2, y = 2.714, z = 2.571$
Pass / Fail	Pass
Comments	The execution of the program takes around 5 minutes to complete. The answers have also been rounded up correctly.

	label	opcode	operand	hex
☐	100	Main	INPUT	5000
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9

AC 0000 (Hex)
IR 5000 (Hex)
MAR 100 (Hex)
MBR 0000 (Hex)
PC 101 (Hex)
INPUT 3 Dec

Test Case 3 — Input a

	label	opcode	operand	hex
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9
☐	10C		INPUT	5000

AC 0003 (Hex)
IR 5000 (Hex)
MAR 102 (Hex)
MBR 0003 (Hex)
PC 103 (Hex)
INPUT 2 Dec

Test Case 3 — Input b

	label	opcode	operand	hex
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9
☐	10C		INPUT	5000

AC 0002 (Hex)
IR 5000 (Hex)
MAR 104 (Hex)
MBR 0002 (Hex)
PC 105 (Hex)
INPUT 1 Dec

Test Case 3 — Input c

	label	opcode	operand	hex
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9
☐	10C		INPUT	5000

AC 0001 (Hex)
IR 5000 (Hex)
MAR 106 (Hex)
MBR 0001 (Hex)
PC 107 (Hex)
INPUT 14 Dec

Test Case 3 — Input d

	label	opcode	operand	hex
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9
☐	10C		INPUT	5000

AC 000E (Hex)
IR 5000 (Hex)
MAR 108 (Hex)
MBR 000E (Hex)
PC 109 (Hex)
INPUT 1 Dec

Test Case 3 — Input e

	label	opcode	operand	hex
☐	101		STORE	a 22B4
☐	102		INPUT	5000
☐	103		STORE	b 22B5
☐	104		INPUT	5000
☐	105		STORE	c 22B6
☐	106		INPUT	5000
☐	107		STORE	d 22B7
☐	108		INPUT	5000
☐	109		STORE	e 22B8
☐	10A		INPUT	5000
☐	10B		STORE	f 22B9
☐	10C		INPUT	5000

AC 0001 (Hex)
IR 5000 (Hex)
MAR 10A (Hex)
MBR 0001 (Hex)
PC 10B (Hex)
INPUT 4 Dec

Test Case 3 — Input f

	label	opcode	operand	hex
☐ 101		STORE	a	22B4
☐ 102		INPUT		5000
☐ 103		STORE	b	22B5
☐ 104		INPUT		5000
☐ 105		STORE	c	22B6
☐ 106		INPUT		5000
☐ 107		STORE	d	22B7
☐ 108		INPUT		5000
☐ 109		STORE	e	22B8
☐ 10A		INPUT		5000
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000

AC 0004 (Hex)
IR 5000 (Hex)
MAR 10C (Hex)
MBR 0004 (Hex)
PC 10D (Hex)
INPUT 2 Dec

Test Case 3 — Input g

	label	opcode	operand	hex
☐ 103		STORE	b	22B5
☐ 104		INPUT		5000
☐ 105		STORE	c	22B6
☐ 106		INPUT		5000
☐ 107		STORE	d	22B7
☐ 108		INPUT		5000
☐ 109		STORE	e	22B8
☐ 10A		INPUT		5000
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000
☐ 10D		STORE	g	22BA
☐ 10E		INPUT		5000

AC 0002 (Hex)
IR 5000 (Hex)
MAR 10E (Hex)
MBR 0002 (Hex)
PC 10F (Hex)
INPUT 18 Dec

Test Case 3 — Input h

	label	opcode	operand	hex
☐ 105		STORE	c	22B6
☐ 106		INPUT		5000
☐ 107		STORE	d	22B7
☐ 108		INPUT		5000
☐ 109		STORE	e	22B8
☐ 10A		INPUT		5000
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000
☐ 10D		STORE	g	22BA
☐ 10E		INPUT		5000
☐ 10F		STORE	h	22BB
☐ 110		INPUT		5000

AC 0012 (Hex)
IR 5000 (Hex)
MAR 110 (Hex)
MBR 0012 (Hex)
PC 111 (Hex)
INPUT 2 Dec

Test Case 3 — Input i

	label	opcode	operand	hex
☐ 107		STORE	d	22B7
☐ 108		INPUT		5000
☐ 109		STORE	e	22B8
☐ 10A		INPUT		5000
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000
☐ 10D		STORE	g	22BA
☐ 10E		INPUT		5000
☐ 10F		STORE	h	22BB
☐ 110		INPUT		5000
☐ 111		STORE	i	22BC
☐ 112		INPUT		5000

AC 0002 (Hex)
IR 5000 (Hex)
MAR 112 (Hex)
MBR 0002 (Hex)
PC 113 (Hex)
INPUT 3 Dec

Test Case 3 — Input j

	label	opcode	operand	hex
☐ 109		STORE	e	22B8
☐ 10A		INPUT		5000
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000
☐ 10D		STORE	g	22BA
☐ 10E		INPUT		5000
☐ 10F		STORE	h	22BB
☐ 110		INPUT		5000
☐ 111		STORE	i	22BC
☐ 112		INPUT		5000
☐ 113		STORE	j	22BD
☐ 114		INPUT		5000

AC 0003 (Hex)
IR 5000 (Hex)
MAR 114 (Hex)
MBR 0003 (Hex)
PC 115 (Hex)
INPUT 5 Dec

Test Case 3 — Input k

	label	opcode	operand	hex
☐ 10B		STORE	f	22B9
☐ 10C		INPUT		5000
☐ 10D		STORE	g	22BA
☐ 10E		INPUT		5000
☐ 10F		STORE	h	22BB
☐ 110		INPUT		5000
☐ 111		STORE	i	22BC
☐ 112		INPUT		5000
☐ 113		STORE	j	22BD
☐ 114		INPUT		5000
☐ 115		STORE	k	22BE
☐ 116		INPUT		5000

AC 0005 (Hex)
IR 5000 (Hex)
MAR 116 (Hex)
MBR 0005 (Hex)
PC 117 (Hex)
INPUT 25 Dec

Test Case 3 — Input l

	label	opcode	operand	hex
☐ 227		STORE	z	22D8
☐ 228		LOAD	z	12D8
☐ 229		OUTPUT		6000
☐ 22A		HALT		7000
☐ 22B	Division	HEX	0	0000
☐ 22C		LOAD	ZERO	12D9
☐ 22D		STORE	quotient	22D3
☐ 22E		STORE	sign_div	22D5
☐ 22F		LOAD	dividend	12D1
☐ 230		SKIPCOND	000	8000
☐ 231		JUMP	Checkd...	9239
☐ 232		LOAD	sign div	12D5

AC 0003 (Hex)
IR 7000 (Hex)
MAR 22A (Hex)
MBR 0003 (Hex)
PC 22B (Hex)
INPUT 25 Dec

OUTPUT

2
3
3

Dec Control

Test Case 3 — Output x, y, z

5. Report / Documentation

Purpose of the program

The purpose of this program is to solve a system of three simultaneous linear equations in three unknowns using Cramer's Rule, implemented entirely in MARIE assembly language. Systems of linear equations appear throughout mathematics, engineering, economics, and computer science — for instance in modelling circuit currents, balancing chemical equations, or computing transformations in computer graphics.

MARIE (Machine Architecture that is Really Intuitive and Easy) provides a minimal yet complete instruction set that mirrors real-world processor design. Implementing a mathematically rich algorithm such as Cramer's Rule in MARIE gives direct experience of how high-level computations break down into fetch–decode–execute cycles, how memory is managed at the instruction level, and how arithmetic operations that are taken for granted in high-level languages must be constructed from primitive additions and subtractions.

Beyond the mathematical goal, the program demonstrates three fundamental assembly programming techniques:

- Subroutine design — encapsulating multiplication and division as callable routines using JnS and Jmpl.
- Memory management — using LOAD and STORE to shuttle values between the single accumulator (AC) and named memory locations for the many intermediate products required by a 3×3 determinant calculation.
- Conditional control flow — using SKIPCOND to detect a zero determinant and branch to an error-output path.

How is Cramer's Rule implemented:

$$A * X = B$$

A: Coefficient square matrix in this case a 3x3 matrix

$$\begin{bmatrix} a & b & c \\ e & f & g \\ i & j & k \end{bmatrix}$$

X: Column vector of variables

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

B: Resultant vector

$$\begin{bmatrix} d \\ h \\ l \end{bmatrix}$$

The linear system of equations is solved by calculating determinants of 3x3 matrices and performing division operation with the determinant values. Three new matrices Ax, Ay and Az are derived where in each the corresponding x, y, z columns are replaced with B.

$$\begin{array}{c} \mathbf{Ax} \\ \begin{bmatrix} d & b & c \\ h & f & g \\ l & j & k \end{bmatrix} \\ x = \frac{dAx}{D} \end{array}$$

$$\begin{array}{c} \mathbf{Bx} \\ \begin{bmatrix} a & d & c \\ e & h & g \\ i & l & k \end{bmatrix} \\ y = \frac{dAy}{D} \end{array}$$

$$\begin{array}{c} \mathbf{Cx} \\ \begin{bmatrix} a & b & d \\ e & f & h \\ i & j & l \end{bmatrix} \\ z = \frac{dAz}{D} \end{array}$$

D is the determinant of the A and Dx, Dy and Dz are the respective determinants of matrices x, y and z.

How are MARIE instructions used:

MARIE provides a minimal instruction set. The table below lists every instruction category used in this program, with a brief explanation and a representative example from the code.

Category	Instructions Used	Purpose in This Program	Example
Data Transfer	LOAD, STORE	Move values between the AC and named memory locations. Essential for handling intermediate products because MARIE has only one general-purpose register.	LOAD afk / STORE D
Arithmetic	ADD, SUBT	Perform addition and subtraction on the AC. All multiplication and division are built on top of these two instructions.	ADD bgi / SUBT agj
Input / Output	INPUT, OUTPUT	Read a value from the user into the AC (INPUT) or write the AC to the output device (OUTPUT).	INPUT / STORE a
Control Flow	JUMP, JnS, Juml	JUMP for unconditional branches (loop tops, loop exits). JnS stores the return address and jumps to a subroutine. Juml reads an address from memory and jumps to it (used for subroutine return).	JnS Multiply / Juml Multiply
Conditional	SKIPCOND	Skip the next instruction based on the sign of the AC: 000 (AC < 0), 400 (AC = 0), 800 (AC > 0). Used in multiplication/division loop counters and in the D = 0 error check.	SKIPCOND 400 (D = 0 check)
Memory Initialisation	DEC, HEX, ORG	Declare variables and constants in the data section. ORG sets the starting address of the program.	Zero, DEC 0 / One, DEC 1

Limitations in MARIE simulator:

- The MARIE simulator only supports addition (ADD) and subtraction (SUBT) directly, thus does not support complex arithmetic operations for example: multiplication/division require iterations and logic whereby the code becomes unnecessarily long and unpractical.
- MARIE Simulator does not support decimal numbers or fractions as it doesn't have support for floating-point arithmetic.
- MARIE simulator has a limited range for numbers from -32768 to 32767.
- The MARIESim.jar takes a lot of time to run due to continuous calling of Multiplication and Division subroutines. This is because the subroutine themselves takes significant time as they perform repeated addition or subtraction.
- The MARIE simulator is constrained by a 12-bit address space, which limits it to 0000 minimum and maximum FFF in hexadecimal.
- There is no debug option/tool to determine how fast and efficient the code executes (run time) or which lines exactly the errors are located (debuggers).

Reflection on what we learned

What have we learnt?

- We learned how to build a multiplication function for multiplying two numbers and then decided how to extend it to multiplying three numbers.
- We learned how to compare the magnitude of two numbers by subtraction, checking which one is greater than the other by using SKIPCOND (000/400/800).
- We learned that the AC is found in the ALU for MARIE Architecture whereas in the Von Neumann architecture, they are separate. This forces frequent STORE and LOAD pairs whenever more than one value must be retained simultaneously. This constraint heavily influences the memory layout of this program.
- We also found out MARIE simulator doesn't just store your data variables in memory; it stores your instructions as well, treating them both as 16-bit binary numbers.
- We realized that in the MARIE simulator, ORG 100 is used to set the program's starting memory address to 100h (hexadecimal). While not technically required by the simulator itself, this directive is a common teaching convention that mimics real-world legacy systems.
- Passing parameters to subroutines in assembly proved to be a challenge, so we used variables globally when calling the subroutine and running the subroutine.
- We also figured out how to round up and round down a non-exact division using successive subtraction and comparing its remainder.
- When implementing a polarity change of a number in MARIE assembly language, the easiest way to change the sign of a number is to subtract it from zero. This was very useful for handling negative numbers as we had to convert negative variables to their absolute values as well as convert positive results to negative when required.
- We also had to be very careful with naming our variables, making sure not to accidentally write multiple subroutines using the same name (for example: Loop, Continue). This could not only lead to syntax errors but also logic errors as well.
- To summarise we learned to use restricted resources having many constraints and produce deliverables mimicking real life conditions and complexities, just like in the world of work.

CONCLUSION

Throughout this coursework, learning MARIE assembly language and its simulator was a daunting experience. Guided by the course materials and lectures from our lecturer, **Dr. Razvi Doomun**, we successfully designed Cramer's rule methodology which require designing and implementing two fundamental and non-native arithmetic operations – Multiplication and Division. Developing these subroutines required addressing complex low-level challenges, specifically managing sign handling for negative operands and ensuring precision for decimal outputs. Regardless of the difficulties we faced, the process provided a rewarding insight into how low-level hardware manages high-level logic. Furthermore, we found MARIE.js to be the superior environment for execution due to its efficiency. Overall, building these arithmetic foundations from the ground up significantly enhanced our problem-solving skills and understanding of computer architecture.

REFERENCES

1. Razvi (2022). *Razvi Doomun - ICT 1206Y Computer Org & Arch. Level 1 Semester 2*. [online] Google.com. Available at: <https://sites.google.com/uom.ac.mu/razvi-doomun/ict-1206y-computer-org-arch-level-1-semester-2>.
2. Unit-2 Part-1: BASIC COMPUTER ORGANIZATION AND DESIGN. (n.d.). Available at: <https://vardhaman.org/wp-content/uploads/2021/03/COA-Unit-II-part-1.pdf>.
3. MIPS Assembly Language Supplement (Section 4.14 of the textbook). (n.d.). Available at: <http://www.cs.uni.edu/~fienup/cs1410s14/lectures/Supplement MIPS.pdf>
4. Github.io. (2026). *Run Time Data Structures*. [online] Available at: <https://silcnitc.github.io/run-data-structures.html>.
5. A. (2026). *Arithmetic and Logic Instructions*. [online] ECE353: Introduction to Microprocessor Systems. Available at: <https://ece353.engr.wisc.edu/arm-assembly/alu/>